ENTREGA 1

Asignatura:

CONCEPTOS FUNDAMENTALES DE PROGRAMACIÓN

Docente:

VÍCTOR FABIÁN CASTRO PÉREZ

Estudiantes:

FERNANDO BETANCUR LONDOÑO

KAREN VALENTINA SANCHEZ AZA

CRISTIAN CAMILO VELEZ CAÑON

LUIS ANGEL CHICUE MOYANO

Programa:

INGENIERIA DE SOFTWARE

Institución:

INSTITUCIÓN UNIVERSITARIA POLITÉCNICO GRANCOLOMBIANO

Fecha:

24 DE MARZO DE 2026

Repositorio GitHub:

https://github.com/ccvelez350-sys/cfpProyecto

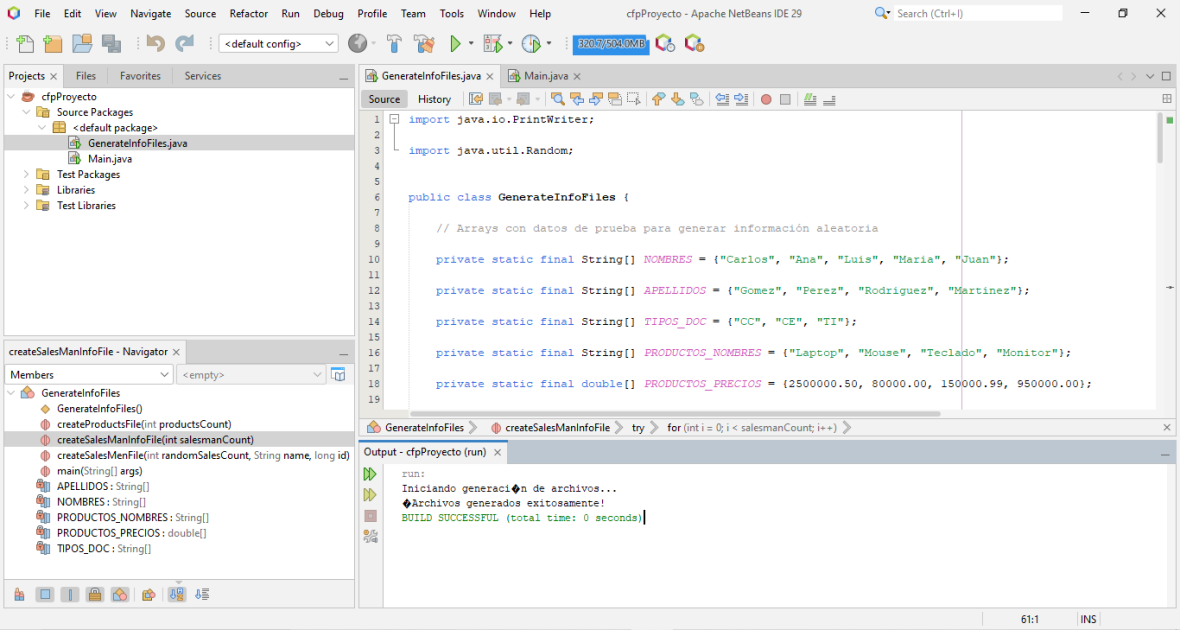
INTRODUCCIÓN

El presente proyecto tiene como objetivo desarrollar un sistema en Java capaz de gestionar, procesar y analizar información relacionada con productos, vendedores y ventas realizadas. Para ello, se implementó una estructura basada en archivos CSV y clases orientadas a objetos que permiten organizar y manipular los datos de manera eficiente.

El sistema se compone de dos partes principales: una herramienta generadora de archivos (GenerateInfoFiles.java), que crea los datos de prueba necesarios para la ejecución, y un módulo de procesamiento (Main.java), encargado de leer los archivos generados, calcular la información correspondiente y producir reportes finales.

A través de este enfoque, el programa automatiza el manejo de ventas, facilita el cálculo del total vendido por cada vendedor, registra la cantidad de productos comercializados y produce reportes ordenados para su posterior análisis. El diseño modular y orientado a objetos permite que el sistema sea comprensible, escalable y adaptable para futuras mejoras.

CODIGO DE GenerateInfoFiles.JAVA




```
import java.io.PrintWriter;

import java.util.Random;

public class GenerateInfoFiles {

    // Arrays con datos de prueba para generar información aleatoria

    private static final String[] NOMBRES = {"Carlos", "Ana", "Luis", "Maria", "Juan"};

    private static final String[] APELLIDOS = {"Gomez", "Perez", "Rodriguez",
"Martinez"};

    private static final String[] TIPOS_DOC = {"CC", "CE", "TI"};

    private static final String[] PRODUCTOS_NOMBRES = {"Laptop", "Mouse",
"Teclado", "Monitor"};

    private static final double[] PRODUCTOS_PRECIOS = {2500000.50, 80000.00,
150000.99, 950000.00};

    public static void main(String[] args) {

        try {

            System.out.println("Iniciando generación de archivos...");

            createProductsFile(PRODUCTOS_NOMBRES.length);

            createSalesManInfoFile(3); // Crear 3 vendedores

            System.out.println("¡Archivos generados exitosamente!");
```

```
        } catch (Exception e) { System.err.println("ERROR: " + e.getMessage()); }
    }
    public static void createProductsFile(int productsCount) throws Exception {
        try (PrintWriter writer = new PrintWriter("productos.csv", "UTF-8")) {
            for (int i = 0; i < productsCount; i++) {
                writer.println((i + 1) + ";" + PRODUCTOS_NOMBRES[i] + ";" +
PRODUCTOS_PRECIOS[i]);
            }
        }
    }
    public static void createSalesManInfoFile(int salesmanCount) throws Exception {


        Random rand = new Random();
        try (PrintWriter writer = new PrintWriter("vendedores.csv", "UTF-8")) {
            for (int i = 0; i < salesmanCount; i++) {
                long id = 100000000 + rand.nextInt(900000000);
                String nombre = NOMBRES[rand.nextInt(NOMBRES.length)];
                writer.println(TIPOS_DOC[rand.nextInt(TIPOS_DOC.length)] + ";" + id +
";" + nombre + ";" + APELLIDOS[rand.nextInt(APELLIDOS.length)]);
                createSalesMenFile(rand.nextInt(4) + 2, nombre, id);
            }
        }
    }
    public static void createSalesMenFile(int randomSalesCount, String name, long
id) throws Exception {
        Random rand = new Random();
        String fileName = "vendedor_" + id + ".csv";
        try (PrintWriter writer = new PrintWriter(fileName, "UTF-8")) {
```

```
            writer.println(TIPOS_DOC[rand.nextInt(TIPOS_DOC.length)] + ";" + id);
            for (int i = 0; i < randomSalesCount; i++) {
                writer.println((rand.nextInt(PRODUCTOS_NOMBRES.length) + 1) + ";" +
(rand.nextInt(10) + 1) + ";");
            }
        }
    }
}
```

Genera los siguientes archivos al correr GenerateInfoFiles.java

productos.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_243168858.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_362759418.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_581599882.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedores.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB

En el **GenerateInfoFiles.java**

Este es un archivo para **generar automáticamente los datos de prueba**.

Funcionalidad principal

Crea todos los archivos necesarios para que Main.java tenga datos reales.

Incluye 3 funciones:

createProductsFile() este genera productos.csv

Incluye:

- ID del producto
- Nombre del producto
- Precio

Nombre	Fecha de modificación	Tipo	Tamaño
reporte_productos.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
reporte_vendedores.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
vendedor_243168858.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_362759418.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_581599882.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedores.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
productos.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB

```
1;Laptop;2500000.5
2;Mouse;80000.0
3;Teclado;150000.99
4;Monitor;950000.0
```

createSalesManInfoFile() Este genera vendedores.csv

Incluye:

- tipoDoc
- numDoc
- nombre
- apellido

Y además crea automáticamente **un archivo para cada vendedor**: vendedor_<numDoc>.csv

Nombre	Fecha de modificación	Tipo	Tamaño
reporte_productos.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
reporte_vendedores.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
vendedor_243168858.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_362759418.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_581599882.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedores.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
productos.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB

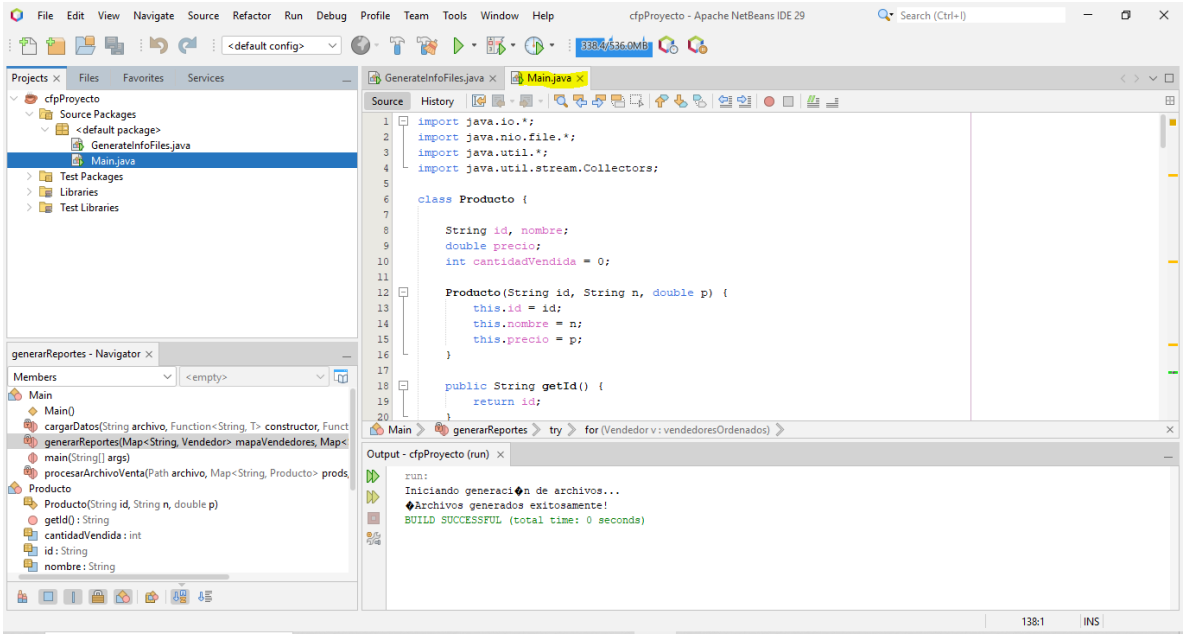
```
CC;362759418;Juan;Gomez
CC;581599882;Maria;Martinez
CE;243168858;Luis;Gomez
```

createSalesMenFile() Genera vendedor_#########.csv

Nombre	Fecha de modificación	Tipo	Tamaño
reporte_productos.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
reporte_vendedores.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
vendedor_243168858.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_362759418.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_581599882.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedores.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
productos.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB

```
CC;243168858
3;3;
2;6;
1;8;
2;4;
2;5;
```

CODIGO DE Main.JAVA




```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.stream.Collectors;
class Producto {
    String id, nombre;
    double precio;
    int cantidadVendida = 0;
    Producto(String id, String n, double p) {
        this.id = id;
        this.nombre = n;
        this.precio = p;
    }
    public String getId() {
        return id;
    }
}
```

```
class Vendedor {
    String tipoDoc, numDoc, nombres, apellidos;
    double ventasTotales = 0.0;
    Vendedor(String td, String nd, String n, String a) {
        this.tipoDoc = td;
        this.numDoc = nd;
        this.nombres = n;
        this.apellidos = a;
    }
    public String getNumDoc() {
        return numDoc;
    }

public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Iniciando generación de archivos...");

            Map<String, Producto> mapaProductos = cargarDatos(
                    "productos.csv",
                    linea -> {
                        String[] d = linea.split(";");
                        return new Producto(d[0], d[1], Double.parseDouble(d[2]));
                    },
                    Producto::getId
            );
            Map<String, Vendedor> mapaVendedores = cargarDatos(
                    "vendedores.csv",
```

```
                linea -> {
                    String[] d = linea.split(";");
                    return new Vendedor(d[0], d[1], d[2], d[3]);
                },
                Vendedor::getNumDoc
            );
            Files.walk(Paths.get("."))
                .filter(path -> path.getFileName().toString().startsWith("vendedor_"))
                .forEach(path -> procesarArchivoVenta(path, mapaProductos,
mapaVendedores));
            generarReportes(mapaVendedores, mapaProductos);
            System.out.println("¡Archivos generados exitosamente!")
        } catch (Exception e) {
            System.err.println("ERROR: " + e.getMessage());
        }
    }
    private static <T> Map<String, T> cargarDatos(
            String archivo,
            java.util.function.Function<String, T> constructor,
            java.util.function.Function<T, String> getKey) throws IOException {
        return Files.lines(Paths.get(archivo))
                .map(constructor)
                .collect(Collectors.toMap(getKey, item -> item));
    }
    private static void procesarArchivoVenta(
            Path archivo,
            Map<String, Producto> prods,
            Map<String, Vendedor> vends) {
        try {
```

```
        List<String> lineas = Files.readAllLines(archivo);
        String idVendedor = lineas.get(0).split(";")[1];
        Vendedor vendedor = vends.get(idVendedor);
        if (vendedor == null)
            return;

        for (int i = 1; i < lineas.size(); i++) {
            String[] datos = lineas.get(i).split(";");
            Producto producto = prods.get(datos[0]);
            int cantidad = Integer.parseInt(datos[1]);
            if (producto != null) {
                vendedor.ventasTotales += producto.precio * cantidad;
                producto.cantidadVendida += cantidad;
            }
        }
    } catch (Exception e) {
        System.err.println("ADVERTENCIA: " + archivo.getFileName());
    }
}
private static void generarReportes(
        Map<String, Vendedor> mapaVendedores,
        Map<String, Producto> mapaProductos) throws IOException {
    List<Vendedor> vendedoresOrdenados = mapaVendedores.values().stream()
            .sorted(Comparator.comparingDouble(v -> -v.ventasTotales))
            .collect(Collectors.toList());

    try (PrintWriter writer = new PrintWriter("reporte_vendedores.csv")) {
        for (Vendedor v : vendedoresOrdenados) {
```

```
            writer.printf("%s %s;%.2f\n", v.nombres, v.apellidos, v.ventasTotales);
        }
    }
    List<Producto> productosOrdenados = mapaProductos.values().stream()
            .sorted(Comparator.comparingInt(p -> -p.cantidadVendida))
            .collect(Collectors.toList());
    try (PrintWriter writer = new PrintWriter("reporte_productos.csv")) {
        for (Producto p : productosOrdenados) {
            writer.printf("%s;%.2f\n", p.nombre, p.precio);
        }
    }
  }
}
```

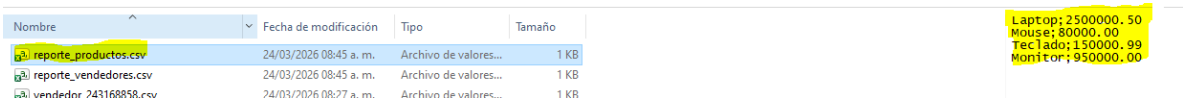
Cuando Corremos el Main.Java nos genera los siguientes archivos.

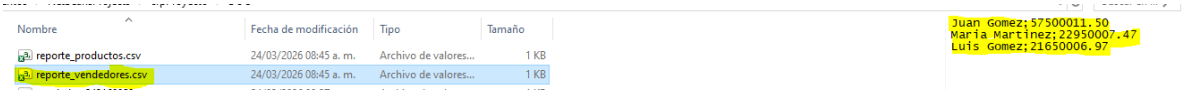
productos.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
reporte_productos.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
reporte_vendedores.csv	24/03/2026 08:45 a. m.	Archivo de valores...	1 KB
vendedor_243168858.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_362759418.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedor_581599882.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB
vendedores.csv	24/03/2026 08:27 a. m.	Archivo de valores...	1 KB

Funcionalidad principal

1. Carga los productos desde productos.csv.
2. Carga los vendedores desde vendedores.csv.
3. Recorre todos los archivos vendedor_XXXX.csv.
4. **Procesa las ventas:**
    - Suma ventas al vendedor.
    - Suma cantidad vendida al producto.
5. **Genera reportes finales:**

- reporte_vendedores.csv: ordenado de mayor a menor ventas.
- reporte_productos.csv: ordenado de más vendido a menos vendido.





IDEAS PARA MEJORAR EL CODIGO

El sistema desarrollado, a pesar de que cumple el objetivo fundamental de gestionar y analizar información de ventas a través de archivos CSV, permite realizar una serie de mejoras para aumentar la funcionalidad, la organización y la calidad del código.

- Primero, se sugeriría incluir encabezados en los archivos CSV generados por el sistema, principalmente en los reportes finales. Por el momento, los archivos CSV generados presentan sólo datos, los cuales no incluyen títulos de columna. Con esto, cabría incluir encabezados como "Nombre del vendedor", "Total de ventas", "Producto", "Precio" y "Cantidad vendida" que permitirían relacionar mejor la información presentada.
- De la misma manera, y aunque el programa sí calcula la cantidad de productos vendidos, ésta no se incluye en el report final de productos. Con ello, la cantidad vendida permitiría realizar un análisis más detallado de cómo se comportan las ventas y facilita la toma de decisiones bajo incidencia de datos.
- Una mejora de importancia relevante sería incorporar también la fecha de generación del reporte en los archivos finales, lo cual permitiría que se lleve una relación cronológica sobre la información generada, facilitando la organización de los resultados y su uso en futuras iteraciones de análisis.
- También podría incorporarse mejoras respecto al manejo de errores del sistema. De momento, el programa tiene una estructura que contempla la captura de excepciones de forma general, pero se podría mejorar

implementando mensajes con mayores niveles de detalle cuando se produzca un problema, tales como archivos que no se hallan en el sistema o errores en el formato de los datos, lo que podría ayudar con la detección de errores y mejorar la experiencia del usuario.

- Desde el punto de vista de la organización del código, el hecho de agrupar las clases en paquetes en función de la función ejercida por éstas resultaría interesante. Por ejemplo, agrupar las clases del modelo (Producto, Vendedor) en un paquete, la lógica del procesamiento de datos en un paquete, etc. Puedes establecer esta agrupación para ayudar a la organización del proyecto y la posibilidad de mejorar el mismo en el futuro o de ampliarlo.
- Por último, se podría dar la oportunidad de ampliar el sistema para permitir el ingreso de datos de forma manual o a través de archivos externos generados por el usuario, y no depender únicamente de datos generados automáticamente.
- Por último, el proyecto podría evolucionar incluyendo nuevas funciones y mejoras tales como la integración de una interfaz gráfica para el usuario, conexión con bases de datos, generación de reportes en PDF o Excel, gráficos con estadísticas de ventas y filtros para búsquedas de datos por fechas o productos. De igual forma, podría transformarse en un sistema aún más útil y adaptado a las necesidades empresariales.

CONCLUSIÓN

El proyecto desarrollado cumple satisfactoriamente con el propósito de gestionar y analizar información de ventas mediante archivos CSV, aplicando principios de programación orientada a objetos y manejo de archivos en Java. La separación de responsabilidades entre las clases permite una ejecución clara: mientras GenerateInfoFiles.java crea los datos necesarios, Main.java procesa la información y genera reportes útiles y estructurados.

Gracias a esta arquitectura, el sistema automatiza el cálculo de ventas por vendedor, registra el comportamiento de los productos vendidos y produce reportes ordenados que facilitan la interpretación de los resultados. En conclusión, la solución demuestra un adecuado dominio del lenguaje Java, una correcta organización lógica y una implementación funcional que cumple con los objetivos planteados.

WEBGRAFIA

Politécnico Grancolombiano. (2026). Conceptos fundamentales de programación.

https://poli.instructure.com

GitHub. (2026). Repositorio del proyecto.

https://github.com/ccvelez350-sys/cfpProyecto

Oracle. (2024). Java Documentation.

https://docs.oracle.com/en/java

Redirecting. (s/f). Instructure.com. Recuperado el 24 de marzo de 2026, de https://poli.instructure.com/courses/93598/discussion_topics/802433

GitHub · Change is constant. GitHub keeps you ahead. (s/f).